DBMS – DATABASE RELATIONSHIP ANALYSIS USING JOINS

Project Title: IKEA E-Commerce Database Management System
Skill Task: DBMS
Topic: Database Relationship Analysis Using Joins

Objectives

1. Understand relationships between multiple database tables.
2. Implement INNER JOIN, LEFT JOIN, and RIGHT JOIN.
3. Retrieve complete order details by combining relevant tables.
4. Display customer purchase history.
5. Generate multi-table reports using SQL joins.

1. INTRODUCTION TO SQL JOINS

1.1 What Is a Join?

A database stores related data in separate tables to avoid repetition. A join combines rows from two or more tables by comparing a common column, such as Customer_ID, that appears in both tables. The comparison is written in the ON clause.

1.2 Why Joins Are Useful in an E-Commerce Database

The order table is written as "Order" in this record because ORDER is an Oracle reserved word and the table name must be quoted. If your table has a different name, such as Orders, use that name in the queries instead. In the IKEA database, customer details are in Customer, each purchase is in Order, the products bought are in Order_Item and Product, and the payment details are in Payment. No single table answers a question such as "Which products did this customer buy, and was the payment completed?". Joins connect the tables through their matching keys: Customer_ID links Customer to Order, Order_ID links Order to Order_Item and Payment, and Product_ID links Order_Item to Product.

1.3 Types of Joins Used

INNER JOIN returns only matching rows. LEFT JOIN keeps every row from the left table and shows NULL for missing matches on the right. RIGHT JOIN keeps every row from the right table and shows NULL for missing matches on the left. Each is demonstrated in this record.

2. INNER JOIN

INNER JOIN returns only the rows that satisfy the join condition. A row that has no match in the other table is not shown.

2.1 Customer and Order

Customer_ID in both tables connects each order to the customer who placed it. Customers 1005 and 1006 have no orders, so they are not listed.

Query 2.1: Customers with Matching Orders

```
SELECT c.Customer_ID, c.Customer_Name,
o.Order_ID, o.Order_Date
FROM Customer c
INNER JOIN "Order" o
ON c.Customer_ID = o.Customer_ID
ORDER BY o.Order_ID;
```

```
SQL> SELECT c.Customer_ID, c.Customer_Name,
  2  o.Order_ID, o.Order_Date
  3  FROM Customer c
  4  INNER JOIN "Order" o
  5  ON c.Customer_ID = o.Customer_ID
  6  ORDER BY o.Order_ID;

CUSTOMER_ID CUSTOMER_NAME ORDER_ID ORDER_DATE
----------- ------------ -------- ----------
       1001 Arun Kumar        5001 02-OCT-26
       1001 Arun Kumar        5002 09-OCT-26
       1002 Divya Lakshmi     5003 12-OCT-26
       1003 Karthik Raja      5004 15-OCT-26
       1004 Meena Priya       5005 18-OCT-26
       1004 Meena Priya       5006 22-OCT-26

6 rows selected.
```

2.2 Order and Order_Item

An order can contain several items, so an Order_ID appears once for every item in it. Order 5001 has two items and therefore two rows.

Query 2.2: Items Belonging to Each Order

```
SELECT o.Order_ID, oi.Order_Item_ID,
oi.Product_ID, oi.Quantity, oi.Unit_Price
FROM "Order" o
INNER JOIN Order_Item oi
ON o.Order_ID = oi.Order_ID
ORDER BY o.Order_ID, oi.Order_Item_ID;
```

```
SQL> SELECT o.Order_ID, oi.Order_Item_ID,
  2  oi.Product_ID, oi.Quantity, oi.Unit_Price
  3  FROM "Order" o
  4  INNER JOIN Order_Item oi
  5  ON o.Order_ID = oi.Order_ID
  6  ORDER BY o.Order_ID, oi.Order_Item_ID;

ORDER_ID ORDER_ITEM_ID PRODUCT_ID QUANTITY UNIT_PRICE
-------- ------------- ---------- -------- ----------
    5001          7001        101        1       4999
    5001          7002        103        2       2499
    5002          7003        107        2       1499
    5002          7004        110        3        799
    5003          7005        102        1      12999
    5004          7006        105        1       5999
    5004          7007        108        2       1999
    5005          7008        104        1       6999
    5006          7009        101        1       4999
    5006          7010        109        1       8999

10 rows selected.
```

2.3 Order_Item and Product

Product_ID links each ordered item to its product, so the product name can be shown instead of only the number.

Query 2.3: Product Details of Each Order Item

```
SELECT oi.Order_ID, p.Product_ID, p.Product_Name,
oi.Quantity, oi.Unit_Price
FROM Order_Item oi
INNER JOIN Product p
ON oi.Product_ID = p.Product_ID
ORDER BY oi.Order_ID, p.Product_ID;
```

```
SQL> SELECT oi.Order_ID, p.Product_ID, p.Product_Name,
  2  oi.Quantity, oi.Unit_Price
  3  FROM Order_Item oi
  4  INNER JOIN Product p
  5  ON oi.Product_ID = p.Product_ID
  6  ORDER BY oi.Order_ID, p.Product_ID;

ORDER_ID PRODUCT_ID PRODUCT_NAME           QUANTITY UNIT_PRICE
-------- ---------- ---------------------- -------- ----------
    5001        101 KALLAX Shelf Unit             1       4999
    5001        103 LACK Coffee Table             2       2499
    5002        107 FADO Table Lamp               2       1499
    5002        110 FEJKA Artificial Plant        3        799
    5003        102 MALM Bed Frame                1      12999
    5004        105 MICKE Desk                    1       5999
    5004        108 MULIG Clothes Rack            2       1999
    5005        104 POANG Armchair                1       6999
    5006        101 KALLAX Shelf Unit             1       4999
    5006        109 APPLARO Outdoor Table         1       8999

10 rows selected.
```

2.4 Order and Payment

Only orders that have a payment record appear. Order 5006 has no payment record, so it is missing from this result.

Query 2.4: Orders with Their Payments

```
SELECT o.Order_ID, o.Order_Date, p.Payment_ID,
p.Payment_Method, p.Payment_Status, p.Amount
FROM "Order" o
INNER JOIN Payment p
ON o.Order_ID = p.Order_ID
ORDER BY o.Order_ID;
```

```
SQL> SELECT o.Order_ID, o.Order_Date, p.Payment_ID,
  2  p.Payment_Method, p.Payment_Status, p.Amount
  3  FROM "Order" o
  4  INNER JOIN Payment p
  5  ON o.Order_ID = p.Order_ID
  6  ORDER BY o.Order_ID;

ORDER_ID ORDER_DATE PAYMENT_ID PAYMENT_METHOD PAYMENT_STATUS AMOUNT
-------- ---------- ---------- -------------- -------------- ------
    5001 02-OCT-26        9001 UPI            Paid             9997
    5002 09-OCT-26        9002 Credit Card    Paid             5395
    5003 12-OCT-26        9003 Net Banking    Paid            12999
    5004 15-OCT-26        9004 COD            Pending          9997
    5005 18-OCT-26        9005 UPI            Paid             6999

5 rows selected.
```

2.5 Multi-table INNER JOIN

Each INNER JOIN adds one more table through its key. The result has one row per order item. Order 5006 (no payment) is excluded because INNER JOIN returns only rows that satisfy every join

condition. Aliases such as Cust_ID and Qty keep the columns narrow; the customer name and order total are shown in later sections.

Query 2.5: Customer, Order, Order_Item, Product and Payment

```
SELECT c.Customer_ID AS Cust_ID, o.Order_ID,
o.Order_Date, p.Product_ID AS Prod_ID,
p.Product_Name, oi.Quantity AS Qty,
pay.Payment_Method AS Pay_Method,
pay.Payment_Status AS Pay_Status
FROM Customer c
INNER JOIN "Order" o
ON c.Customer_ID = o.Customer_ID
INNER JOIN Order_Item oi
ON o.Order_ID = oi.Order_ID
INNER JOIN Product p
ON oi.Product_ID = p.Product_ID
INNER JOIN Payment pay
ON o.Order_ID = pay.Order_ID
ORDER BY o.Order_ID, p.Product_ID;
```

```
SQL> SELECT c.Customer_ID AS Cust_ID, o.Order_ID,
  2  o.Order_Date, p.Product_ID AS Prod_ID,
  3  p.Product_Name, oi.Quantity AS Qty,
  4  pay.Payment_Method AS Pay_Method,
  5  pay.Payment_Status AS Pay_Status
  6  FROM Customer c
  7  INNER JOIN "Order" o
  8  ON c.Customer_ID = o.Customer_ID
  9  INNER JOIN Order_Item oi
 10  ON o.Order_ID = oi.Order_ID
 11  INNER JOIN Product p
 12  ON oi.Product_ID = p.Product_ID
 13  INNER JOIN Payment pay
 14  ON o.Order_ID = pay.Order_ID
 15  ORDER BY o.Order_ID, p.Product_ID;

CUST_ID ORDER_ID ORDER_DATE PROD_ID PRODUCT_NAME          QTY PAY_METHOD  PAY_STATUS
------- -------- ---------- ------- ---------------------- --- ----------- ----------
   1001     5001 02-OCT-26      101 KALLAX Shelf Unit        1 UPI         Paid
   1001     5001 02-OCT-26      103 LACK Coffee Table        2 UPI         Paid
   1001     5002 09-OCT-26      107 FADO Table Lamp          2 Credit Card Paid
   1001     5002 09-OCT-26      110 FEJKA Artificial Plant   3 Credit Card Paid
   1002     5003 12-OCT-26      102 MALM Bed Frame           1 Net Banking Paid
   1003     5004 15-OCT-26      105 MICKE Desk               1 COD         Pending
   1003     5004 15-OCT-26      108 MULIG Clothes Rack       2 COD         Pending
   1004     5005 18-OCT-26      104 POANG Armchair           1 UPI         Paid

8 rows selected.
```

3. LEFT JOIN

LEFT JOIN returns every row from the left table (the table written before the JOIN keyword) and the matching rows from the right table. When there is no match, the right-hand columns contain NULL, which SQL*Plus displays as blank.

3.1 Customer and Order

Customer is the left table, so every customer is kept. Customers 1005 and 1006 have no orders, so their Order_ID and Order_Date are NULL (blank).

Query 3.1: All Customers Including Those Without Orders

```
SELECT c.Customer_ID, c.Customer_Name,
o.Order_ID, o.Order_Date
FROM Customer c
LEFT JOIN "Order" o
ON c.Customer_ID = o.Customer_ID
ORDER BY c.Customer_ID, o.Order_ID;
```

```
SQL> SELECT c.Customer_ID, c.Customer_Name,
  2  o.Order_ID, o.Order_Date
  3  FROM Customer c
  4  LEFT JOIN "Order" o
  5  ON c.Customer_ID = o.Customer_ID
  6  ORDER BY c.Customer_ID, o.Order_ID;

CUSTOMER_ID CUSTOMER_NAME ORDER_ID ORDER_DATE
----------- ------------ -------- ----------
       1001 Arun Kumar        5001 02-OCT-26
       1001 Arun Kumar        5002 09-OCT-26
       1002 Divya Lakshmi     5003 12-OCT-26
       1003 Karthik Raja      5004 15-OCT-26
       1004 Meena Priya       5005 18-OCT-26
       1004 Meena Priya       5006 22-OCT-26
       1005 Suresh Babu
       1006 Anitha Devi

8 rows selected.
```

3.2 Order and Payment

Every order is shown. Order 5006 has no matching payment, so the payment columns are NULL.

Query 3.2: All Orders Including Those Without Payment

```
SELECT o.Order_ID, o.Order_Date, p.Payment_ID,
p.Payment_Method, p.Payment_Status
FROM "Order" o
LEFT JOIN Payment p
ON o.Order_ID = p.Order_ID
ORDER BY o.Order_ID;
```

```
SQL> SELECT o.Order_ID, o.Order_Date, p.Payment_ID,
  2  p.Payment_Method, p.Payment_Status
  3  FROM "Order" o
  4  LEFT JOIN Payment p
  5  ON o.Order_ID = p.Order_ID
  6  ORDER BY o.Order_ID;

ORDER_ID ORDER_DATE PAYMENT_ID PAYMENT_METHOD PAYMENT_STATUS
-------- ---------- ---------- -------------- --------------
    5001 02-OCT-26        9001 UPI            Paid
    5002 09-OCT-26        9002 Credit Card    Paid
    5003 12-OCT-26        9003 Net Banking    Paid
    5004 15-OCT-26        9004 COD            Pending
    5005 18-OCT-26        9005 UPI            Paid
    5006 22-OCT-26

6 rows selected.
```

3.3 Product and Order_Item

Every product is listed. Product 101 appears twice because it was ordered in two orders, and product 106 (INGOLF Dining Chair) appears once with NULL values because it was never ordered.

Query 3.3: All Products Including Products Never Ordered

```
SELECT p.Product_ID, p.Product_Name,
oi.Order_ID, oi.Quantity
FROM Product p
LEFT JOIN Order_Item oi
ON p.Product_ID = oi.Product_ID
ORDER BY p.Product_ID, oi.Order_ID;
```

```
SQL> SELECT p.Product_ID, p.Product_Name,
  2  oi.Order_ID, oi.Quantity
  3  FROM Product p
  4  LEFT JOIN Order_Item oi
  5  ON p.Product_ID = oi.Product_ID
  6  ORDER BY p.Product_ID, oi.Order_ID;

PRODUCT_ID PRODUCT_NAME           ORDER_ID QUANTITY
---------- ---------------------- -------- --------
       101 KALLAX Shelf Unit          5001        1
       101 KALLAX Shelf Unit          5006        1
       102 MALM Bed Frame             5003        1
       103 LACK Coffee Table          5001        2
       104 POANG Armchair             5005        1
       105 MICKE Desk                 5004        1
       106 INGOLF Dining Chair
       107 FADO Table Lamp            5002        2
       108 MULIG Clothes Rack         5004        2
       109 APPLARO Outdoor Table      5006        1
       110 FEJKA Artificial Plant     5002        3

11 rows selected.
```

3.4 Multi-table LEFT JOIN Report

Customer is the parent table, and Order and Payment are joined with LEFT JOIN, so no customer or order is lost. Customers without orders show NULL for Order_ID and Payment_Status, and order 5006

shows NULL for Payment_Status because it has no payment. An unmatched right-hand table always produces NULL in all of its columns.

Query 3.4: Customer Order and Payment Status Report

```
SELECT c.Customer_ID, c.Customer_Name,
o.Order_ID, p.Payment_Status
FROM Customer c
LEFT JOIN "Order" o
ON c.Customer_ID = o.Customer_ID
LEFT JOIN Payment p
ON o.Order_ID = p.Order_ID
ORDER BY c.Customer_ID, o.Order_ID;
```

```
SQL> SELECT c.Customer_ID, c.Customer_Name,
  2  o.Order_ID, p.Payment_Status
  3  FROM Customer c
  4  LEFT JOIN "Order" o
  5  ON c.Customer_ID = o.Customer_ID
  6  LEFT JOIN Payment p
  7  ON o.Order_ID = p.Order_ID
  8  ORDER BY c.Customer_ID, o.Order_ID;

CUSTOMER_ID CUSTOMER_NAME ORDER_ID PAYMENT_STATUS
----------- ------------- -------- --------------
       1001 Arun Kumar        5001 Paid
       1001 Arun Kumar        5002 Paid
       1002 Divya Lakshmi     5003 Paid
       1003 Karthik Raja      5004 Pending
       1004 Meena Priya       5005 Paid
       1004 Meena Priya       5006
       1005 Suresh Babu
       1006 Anitha Devi

8 rows selected.
```

4. RIGHT JOIN

RIGHT JOIN returns every row from the right table (the table written after the JOIN keyword) and the matching rows from the left table. In each example below the table order is chosen so that the table whose rows must be preserved is on the right.

4.1 Customer and Order

Customer is the right table, so all customers are preserved, including customers 1005 and 1006 with NULL Order_ID. This gives the same rows as the LEFT JOIN in 3.1 with the table order reversed.

Query 4.1: Order RIGHT JOIN Customer

```
SELECT c.Customer_ID, c.Customer_Name, o.Order_ID
FROM "Order" o
RIGHT JOIN Customer c
ON o.Customer_ID = c.Customer_ID
ORDER BY c.Customer_ID, o.Order_ID;
```

```
SQL> SELECT c.Customer_ID, c.Customer_Name, o.Order_ID
  2  FROM "Order" o
  3  RIGHT JOIN Customer c
  4  ON o.Customer_ID = c.Customer_ID
  5  ORDER BY c.Customer_ID, o.Order_ID;

CUSTOMER_ID CUSTOMER_NAME ORDER_ID
----------- ------------ --------
       1001 Arun Kumar        5001
       1001 Arun Kumar        5002
       1002 Divya Lakshmi     5003
       1003 Karthik Raja      5004
       1004 Meena Priya       5005
       1004 Meena Priya       5006
       1005 Suresh Babu
       1006 Anitha Devi

8 rows selected.
```

4.2 Order and Payment

Order is the right table, so every order is preserved. Order 5006 has no payment, so Payment_ID, Payment_Method and Payment_Status are NULL. Putting Payment on the left and Order on the right is what makes the unpaid order visible.

Query 4.2: Payment RIGHT JOIN Order

```
SELECT o.Order_ID, p.Payment_ID,
p.Payment_Method, p.Payment_Status
FROM Payment p
RIGHT JOIN "Order" o
ON p.Order_ID = o.Order_ID
ORDER BY o.Order_ID;
```

```
SQL> SELECT o.Order_ID, p.Payment_ID,
  2  p.Payment_Method, p.Payment_Status
  3  FROM Payment p
  4  RIGHT JOIN "Order" o
  5  ON p.Order_ID = o.Order_ID
  6  ORDER BY o.Order_ID;

ORDER_ID PAYMENT_ID PAYMENT_METHOD PAYMENT_STATUS
-------- ---------- -------------- --------------
    5001       9001 UPI            Paid
    5002       9002 Credit Card    Paid
    5003       9003 Net Banking    Paid
    5004       9004 COD            Pending
    5005       9005 UPI            Paid
    5006

6 rows selected.
```

4.3 Product and Order_Item

Product is the right table, so all products are preserved. Product 106 has no order item, so its Order_ID and Quantity are NULL.

Query 4.3: Order_Item RIGHT JOIN Product

```
SELECT p.Product_ID, p.Product_Name,
oi.Order_ID, oi.Quantity
FROM Order_Item oi
RIGHT JOIN Product p
ON oi.Product_ID = p.Product_ID
ORDER BY p.Product_ID, oi.Order_ID;
```

```
SQL> SELECT p.Product_ID, p.Product_Name,
  2  oi.Order_ID, oi.Quantity
  3  FROM Order_Item oi
  4  RIGHT JOIN Product p
  5  ON oi.Product_ID = p.Product_ID
  6  ORDER BY p.Product_ID, oi.Order_ID;

PRODUCT_ID PRODUCT_NAME           ORDER_ID QUANTITY
---------- ---------------------- -------- --------
       101 KALLAX Shelf Unit          5001        1
       101 KALLAX Shelf Unit          5006        1
       102 MALM Bed Frame             5003        1
       103 LACK Coffee Table          5001        2
       104 POANG Armchair             5005        1
       105 MICKE Desk                 5004        1
       106 INGOLF Dining Chair
       107 FADO Table Lamp            5002        2
       108 MULIG Clothes Rack         5004        2
       109 APPLARO Outdoor Table      5006        1
       110 FEJKA Artificial Plant     5002        3

11 rows selected.
```

4.4 Comparison with LEFT JOIN

LEFT JOIN and RIGHT JOIN do the same job from opposite sides. A RIGHT JOIN can always be rewritten as a LEFT JOIN by swapping the table order, so most programmers prefer LEFT JOIN for readability.

Point	LEFT JOIN	RIGHT JOIN
Preserved table	The table before the JOIN keyword (left)	The table after the JOIN keyword (right)
Unmatched rows	Left rows kept; right columns are NULL	Right rows kept; left columns are NULL
Example	Customer c LEFT JOIN "Order" o	"Order" o RIGHT JOIN Customer c
Same result as	Swapped RIGHT JOIN	Swapped LEFT JOIN

5. RETRIEVE COMPLETE ORDER DETAILS

Order-level information (Order_ID, Order_Date, Total_Amount, payment details) occurs once per order. Order-item-level information (product, Quantity, Unit_Price, line total) occurs once for every product in the order. When an order has several items, the order-level values naturally repeat on each item row, so the result must be read as one row per order item, not one row per order.

5.1 Order Item Details

The line total is calculated as Quantity × Unit_Price for each item. The order total is deliberately not repeated here, because adding it on every item row could be mistaken for the amount of each item.

Query 5.1: One Row per Order Item

```
SELECT o.Order_ID, o.Order_Date, c.Customer_Name,
p.Product_Name, oi.Quantity, oi.Unit_Price,
oi.Quantity * oi.Unit_Price AS Line_Total
FROM Customer c
INNER JOIN "Order" o
ON c.Customer_ID = o.Customer_ID
INNER JOIN Order_Item oi
ON o.Order_ID = oi.Order_ID
INNER JOIN Product p
ON oi.Product_ID = p.Product_ID
ORDER BY o.Order_ID, p.Product_ID;
```

```
SQL> SELECT o.Order_ID, o.Order_Date, c.Customer_Name,
  2  p.Product_Name, oi.Quantity, oi.Unit_Price,
  3  oi.Quantity * oi.Unit_Price AS Line_Total
  4  FROM Customer c
  5  INNER JOIN "Order" o
  6  ON c.Customer_ID = o.Customer_ID
  7  INNER JOIN Order_Item oi
  8  ON o.Order_ID = oi.Order_ID
  9  INNER JOIN Product p
 10  ON oi.Product_ID = p.Product_ID
 11  ORDER BY o.Order_ID, p.Product_ID;

ORDER_ID ORDER_DATE CUSTOMER_NAME PRODUCT_NAME           QUANTITY UNIT_PRICE LINE_TOTAL
-------- ---------- ------------- ---------------------- -------- ---------- ----------
    5001 02-OCT-26  Arun Kumar    KALLAX Shelf Unit             1       4999       4999
    5001 02-OCT-26  Arun Kumar    LACK Coffee Table             2       2499       4998
    5002 09-OCT-26  Arun Kumar    FADO Table Lamp               2       1499       2998
    5002 09-OCT-26  Arun Kumar    FEJKA Artificial Plant        3        799       2397
    5003 12-OCT-26  Divya Lakshmi MALM Bed Frame                1      12999      12999
    5004 15-OCT-26  Karthik Raja  MICKE Desk                    1       5999       5999
    5004 15-OCT-26  Karthik Raja  MULIG Clothes Rack            2       1999       3998
    5005 18-OCT-26  Meena Priya   POANG Armchair                1       6999       6999
    5006 22-OCT-26  Meena Priya   KALLAX Shelf Unit             1       4999       4999
    5006 22-OCT-26  Meena Priya   APPLARO Outdoor Table         1       8999       8999

10 rows selected.
```

5.2 Order Details with Customer and Payment

Order_Item is not joined here, so each order appears exactly once and Total_Amount is not duplicated. LEFT JOIN is used for Payment so that unpaid order 5006 is still listed. Order 5004 is paid by cash on delivery and is still Pending, so its Payment_Date is NULL.

Query 5.2: One Row per Order with Payment Details

```
SELECT o.Order_ID, c.Customer_ID, c.Customer_Name,
o.Total_Amount, p.Payment_Method AS Pay_Method,
p.Payment_Status AS Pay_Status,
p.Payment_Date AS Pay_Date
FROM Customer c
INNER JOIN "Order" o
ON c.Customer_ID = o.Customer_ID
LEFT JOIN Payment p
ON o.Order_ID = p.Order_ID
ORDER BY o.Order_ID;
```

```
SQL> SELECT o.Order_ID, c.Customer_ID, c.Customer_Name,
  2  o.Total_Amount, p.Payment_Method AS Pay_Method,
  3  p.Payment_Status AS Pay_Status,
  4  p.Payment_Date AS Pay_Date
  5  FROM Customer c
  6  INNER JOIN "Order" o
  7  ON c.Customer_ID = o.Customer_ID
  8  LEFT JOIN Payment p
  9  ON o.Order_ID = p.Order_ID
 10  ORDER BY o.Order_ID;

ORDER_ID CUSTOMER_ID CUSTOMER_NAME TOTAL_AMOUNT PAY_METHOD  PAY_STATUS PAY_DATE
-------- ----------- ------------- ------------ ----------- ---------- ---------
    5001        1001 Arun Kumar            9997 UPI         Paid       02-OCT-26
    5002        1001 Arun Kumar            5395 Credit Card Paid       09-OCT-26
    5003        1002 Divya Lakshmi        12999 Net Banking Paid       12-OCT-26
    5004        1003 Karthik Raja          9997 COD         Pending
    5005        1004 Meena Priya           6999 UPI         Paid       18-OCT-26
    5006        1004 Meena Priya          13998

6 rows selected.
```

6. CUSTOMER PURCHASE HISTORY

6.1 Purchase History of All Customers

Each row is one product purchased by one customer in one order. Payment is joined with LEFT JOIN so that purchases without a payment record are still part of the history.

Query 6.1: Customers, Orders and Products Purchased

```
SELECT c.Customer_ID AS Cust_ID, c.Customer_Name,
o.Order_ID, o.Order_Date, p.Product_Name,
oi.Quantity AS Qty,
oi.Quantity * oi.Unit_Price AS Line_Total,
pay.Payment_Status AS Pay_Status
FROM Customer c
INNER JOIN "Order" o
ON c.Customer_ID = o.Customer_ID
INNER JOIN Order_Item oi
ON o.Order_ID = oi.Order_ID
INNER JOIN Product p
ON oi.Product_ID = p.Product_ID
LEFT JOIN Payment pay
ON o.Order_ID = pay.Order_ID
ORDER BY c.Customer_ID, o.Order_ID, p.Product_ID;
```

```
SQL> SELECT c.Customer_ID AS Cust_ID, c.Customer_Name,
  2  o.Order_ID, o.Order_Date, p.Product_Name,
  3  oi.Quantity AS Qty,
  4  oi.Quantity * oi.Unit_Price AS Line_Total,
  5  pay.Payment_Status AS Pay_Status
  6  FROM Customer c
  7  INNER JOIN "Order" o
  8  ON c.Customer_ID = o.Customer_ID
  9  INNER JOIN Order_Item oi
 10  ON o.Order_ID = oi.Order_ID
 11  INNER JOIN Product p
 12  ON oi.Product_ID = p.Product_ID
 13  LEFT JOIN Payment pay
 14  ON o.Order_ID = pay.Order_ID
 15  ORDER BY c.Customer_ID, o.Order_ID, p.Product_ID;

CUST_ID CUSTOMER_NAME ORDER_ID ORDER_DATE PRODUCT_NAME           QTY LINE_TOTAL PAY_STATUS
------- ------------- -------- ---------- --------------------- --- ---------- ----------
   1001 Arun Kumar        5001 02-OCT-26  KALLAX Shelf Unit       1       4999 Paid
   1001 Arun Kumar        5001 02-OCT-26  LACK Coffee Table       2       4998 Paid
   1001 Arun Kumar        5002 09-OCT-26  FADO Table Lamp         2       2998 Paid
   1001 Arun Kumar        5002 09-OCT-26  FEJKA Artificial Plant  3       2397 Paid
   1002 Divya Lakshmi     5003 12-OCT-26  MALM Bed Frame          1      12999 Paid
   1003 Karthik Raja      5004 15-OCT-26  MICKE Desk              1       5999 Pending
   1003 Karthik Raja      5004 15-OCT-26  MULIG Clothes Rack      2       3998 Pending
   1004 Meena Priya       5005 18-OCT-26  POANG Armchair          1       6999 Paid
   1004 Meena Priya       5006 22-OCT-26  KALLAX Shelf Unit       1       4999
   1004 Meena Priya       5006 22-OCT-26  APPLARO Outdoor Table   1       8999

10 rows selected.
```

6.2 Purchase History of One Customer

The WHERE clause limits the history to one customer. 1001 is only a sample Customer_ID; replace it with an ID that exists in your Customer table. In SQL*Plus you may also write &Customer_ID so that the value is requested when the query runs.

Query 6.2: Purchase History of Customer 1001

```
SELECT o.Order_ID, o.Order_Date, p.Product_Name,
oi.Quantity AS Qty,
oi.Quantity * oi.Unit_Price AS Line_Total,
pay.Payment_Status AS Pay_Status
FROM Customer c
INNER JOIN "Order" o
ON c.Customer_ID = o.Customer_ID
INNER JOIN Order_Item oi
ON o.Order_ID = oi.Order_ID
INNER JOIN Product p
ON oi.Product_ID = p.Product_ID
LEFT JOIN Payment pay
ON o.Order_ID = pay.Order_ID
WHERE c.Customer_ID = 1001
ORDER BY o.Order_ID, p.Product_ID;
```

```
SQL> SELECT o.Order_ID, o.Order_Date, p.Product_Name,
  2  oi.Quantity AS Qty,
  3  oi.Quantity * oi.Unit_Price AS Line_Total,
  4  pay.Payment_Status AS Pay_Status
  5  FROM Customer c
  6  INNER JOIN "Order" o
  7  ON c.Customer_ID = o.Customer_ID
  8  INNER JOIN Order_Item oi
  9  ON o.Order_ID = oi.Order_ID
 10  INNER JOIN Product p
 11  ON oi.Product_ID = p.Product_ID
 12  LEFT JOIN Payment pay
 13  ON o.Order_ID = pay.Order_ID
 14  WHERE c.Customer_ID = 1001
 15  ORDER BY o.Order_ID, p.Product_ID;

ORDER_ID ORDER_DATE PRODUCT_NAME           QTY LINE_TOTAL PAY_STATUS
-------- ---------- ---------------------- --- ---------- ----------
    5001 02-OCT-26  KALLAX Shelf Unit        1       4999 Paid
    5001 02-OCT-26  LACK Coffee Table        2       4998 Paid
    5002 09-OCT-26  FADO Table Lamp          2       2998 Paid
    5002 09-OCT-26  FEJKA Artificial Plant   3       2397 Paid

4 rows selected.
```

6.3 Number of Orders per Purchasing Customer

COUNT with GROUP BY counts the orders of each customer. Every non-aggregated column in the SELECT list also appears in GROUP BY. Only customers who have placed orders are shown, because INNER JOIN is used.

Query 6.3: Order Count per Customer

```
SELECT c.Customer_ID, c.Customer_Name,
COUNT(o.Order_ID) AS Total_Orders
FROM Customer c
INNER JOIN "Order" o
ON c.Customer_ID = o.Customer_ID
GROUP BY c.Customer_ID, c.Customer_Name
ORDER BY c.Customer_ID;
```

```
SQL> SELECT c.Customer_ID, c.Customer_Name,
  2  COUNT(o.Order_ID) AS Total_Orders
  3  FROM Customer c
  4  INNER JOIN "Order" o
  5  ON c.Customer_ID = o.Customer_ID
  6  GROUP BY c.Customer_ID, c.Customer_Name
  7  ORDER BY c.Customer_ID;

CUSTOMER_ID CUSTOMER_NAME TOTAL_ORDERS
----------- ------------ ------------
       1001 Arun Kumar              2
       1002 Divya Lakshmi           1
       1003 Karthik Raja            1
       1004 Meena Priya             2

4 rows selected.
```

Such a report helps an e-commerce company see which customers buy repeatedly, which customers have bought only once, and which customers could be targeted with offers.

7. MULTI-TABLE BUSINESS REPORTS

7.1 Customer Order Summary

LEFT JOIN keeps customers without orders. COUNT(o.Order_ID) ignores NULL values, so those customers correctly show 0 instead of 1.

Query 7.1: Orders Placed by Every Customer

```
SELECT c.Customer_ID, c.Customer_Name,
COUNT(o.Order_ID) AS Total_Orders
FROM Customer c
LEFT JOIN "Order" o
ON c.Customer_ID = o.Customer_ID
GROUP BY c.Customer_ID, c.Customer_Name
ORDER BY c.Customer_ID;
```

```
SQL> SELECT c.Customer_ID, c.Customer_Name,
  2  COUNT(o.Order_ID) AS Total_Orders
  3  FROM Customer c
  4  LEFT JOIN "Order" o
  5  ON c.Customer_ID = o.Customer_ID
  6  GROUP BY c.Customer_ID, c.Customer_Name
  7  ORDER BY c.Customer_ID;

CUSTOMER_ID CUSTOMER_NAME TOTAL_ORDERS
----------- ------------ ------------
       1001 Arun Kumar              2
       1002 Divya Lakshmi           1
       1003 Karthik Raja            1
       1004 Meena Priya             2
       1005 Suresh Babu             0
       1006 Anitha Devi             0

6 rows selected.
```

7.2 Order Payment Summary

The order total is shown beside the payment amount so that they can be compared. Order 5006 has no payment record, so all payment columns are NULL.

Query 7.2: Orders with Payment Information

```
SELECT o.Order_ID, o.Total_Amount, p.Payment_Method AS Pay_Method,
p.Amount AS Paid_Amount,
p.Payment_Status AS Pay_Status
FROM "Order" o
LEFT JOIN Payment p
ON o.Order_ID = p.Order_ID
ORDER BY o.Order_ID;
```

```
SQL> SELECT o.Order_ID, o.Total_Amount, p.Payment_Method AS Pay_Method,
  2  p.Amount AS Paid_Amount,
  3  p.Payment_Status AS Pay_Status
  4  FROM "Order" o
  5  LEFT JOIN Payment p
  6  ON o.Order_ID = p.Order_ID
  7  ORDER BY o.Order_ID;

ORDER_ID TOTAL_AMOUNT PAY_METHOD  PAID_AMOUNT PAY_STATUS
-------- ------------ ----------- ----------- ----------
    5001         9997 UPI                9997 Paid
    5002         5395 Credit Card        5395 Paid
    5003        12999 Net Banking       12999 Paid
    5004         9997 COD                9997 Pending
    5005         6999 UPI                6999 Paid
    5006        13998

6 rows selected.
```

7.3 Product Sales Report

SUM adds the quantity of a product across all orders. Product 106 does not appear because it has never been ordered; it is listed in 8.6.

Query 7.3: Total Quantity Sold per Product

```
SELECT p.Product_ID, p.Product_Name,
SUM(oi.Quantity) AS Total_Qty,
SUM(oi.Quantity * oi.Unit_Price) AS Sales_Value
FROM Product p
INNER JOIN Order_Item oi
ON p.Product_ID = oi.Product_ID
GROUP BY p.Product_ID, p.Product_Name
ORDER BY Total_Qty DESC, p.Product_Name;
```

```
SQL> SELECT p.Product_ID, p.Product_Name,
  2  SUM(oi.Quantity) AS Total_Qty,
  3  SUM(oi.Quantity * oi.Unit_Price) AS Sales_Value
  4  FROM Product p
  5  INNER JOIN Order_Item oi
  6  ON p.Product_ID = oi.Product_ID
  7  GROUP BY p.Product_ID, p.Product_Name
  8  ORDER BY Total_Qty DESC, p.Product_Name;

PRODUCT_ID PRODUCT_NAME           TOTAL_QTY SALES_VALUE
---------- ---------------------- --------- -----------
       110 FEJKA Artificial Plant         3        2397
       107 FADO Table Lamp                2        2998
       101 KALLAX Shelf Unit              2        9998
       103 LACK Coffee Table              2        4998
       108 MULIG Clothes Rack             2        3998
       109 APPLARO Outdoor Table          1        8999
       102 MALM Bed Frame                 1       12999
       105 MICKE Desk                     1        5999
       104 POANG Armchair                 1        6999

9 rows selected.
```

7.4 Customer Purchase Amount Report

The amount aggregated is the item line total (Quantity × Unit_Price) from Order_Item. Summing Order.Total_Amount after joining to Order_Item would count the same order total once for every item

and give a wrong, inflated result. NVL replaces NULL with 0 for customers who have not ordered. The report includes unpaid orders; join Payment and filter on Payment_Status to count only paid orders.

Query 7.4: Total Purchase Amount per Customer

```
SELECT c.Customer_ID, c.Customer_Name,
NVL(SUM(oi.Quantity * oi.Unit_Price), 0) AS Total_Spent
FROM Customer c
LEFT JOIN "Order" o
ON c.Customer_ID = o.Customer_ID
LEFT JOIN Order_Item oi
ON o.Order_ID = oi.Order_ID
GROUP BY c.Customer_ID, c.Customer_Name
ORDER BY c.Customer_ID;
```

```
SQL> SELECT c.Customer_ID, c.Customer_Name,
  2  NVL(SUM(oi.Quantity * oi.Unit_Price), 0) AS Total_Spent
  3  FROM Customer c
  4  LEFT JOIN "Order" o
  5  ON c.Customer_ID = o.Customer_ID
  6  LEFT JOIN Order_Item oi
  7  ON o.Order_ID = oi.Order_ID
  8  GROUP BY c.Customer_ID, c.Customer_Name
  9  ORDER BY c.Customer_ID;

CUSTOMER_ID CUSTOMER_NAME TOTAL_SPENT
----------- ------------ -----------
       1001 Arun Kumar          15392
       1002 Divya Lakshmi       12999
       1003 Karthik Raja         9997
       1004 Meena Priya         20997
       1005 Suresh Babu             0
       1006 Anitha Devi             0

6 rows selected.
```

7.5 Orders Without Matching Payments

After the LEFT JOIN, orders without a payment have NULL in Payment_ID. The condition IS NULL (not = NULL) keeps only those orders.

Query 7.5: Orders with No Payment Record

```
SELECT o.Order_ID, o.Customer_ID, o.Order_Date, o.Total_Amount
FROM "Order" o
LEFT JOIN Payment p
ON o.Order_ID = p.Order_ID
WHERE p.Payment_ID IS NULL
ORDER BY o.Order_ID;
```

```
SQL> SELECT o.Order_ID, o.Customer_ID, o.Order_Date, o.Total_Amount
  2  FROM "Order" o
  3  LEFT JOIN Payment p
  4  ON o.Order_ID = p.Order_ID
  5  WHERE p.Payment_ID IS NULL
  6  ORDER BY o.Order_ID;

ORDER_ID CUSTOMER_ID ORDER_DATE TOTAL_AMOUNT
-------- ----------- ---------- ------------
    5006        1004 22-OCT-26         13998

1 row selected.
```

7.6 Products Never Ordered

Products that have no matching Order_Item row get NULL in the Order_Item columns. In this sample data only INGOLF Dining Chair has never been ordered.

Query 7.6: Products with No Order Item

```
SELECT p.Product_ID, p.Product_Name, p.Price
FROM Product p
LEFT JOIN Order_Item oi
ON p.Product_ID = oi.Product_ID
WHERE oi.Order_ID IS NULL
ORDER BY p.Product_ID;
```

```
SQL> SELECT p.Product_ID, p.Product_Name, p.Price
  2  FROM Product p
  3  LEFT JOIN Order_Item oi
  4  ON p.Product_ID = oi.Product_ID
  5  WHERE oi.Order_ID IS NULL
  6  ORDER BY p.Product_ID;

PRODUCT_ID PRODUCT_NAME        PRICE
---------- ------------------ -----
       106 INGOLF Dining Chair  3999

1 row selected.
```

7.7 Combined Customer, Order, Product and Payment Report

This report combines four tables with INNER JOIN and then adds Payment with LEFT JOIN. Unlike 2.5, it keeps the items of order 5006, whose payment columns are NULL, so management can see purchases that are not yet paid.

Query 7.7: Complete Combined Report

```
SELECT c.Customer_Name, o.Order_ID, p.Product_Name,
oi.Quantity AS Qty,
oi.Quantity * oi.Unit_Price AS Line_Total,
pay.Payment_Method AS Pay_Method,
pay.Payment_Status AS Pay_Status
FROM Customer c
INNER JOIN "Order" o
ON c.Customer_ID = o.Customer_ID
INNER JOIN Order_Item oi
ON o.Order_ID = oi.Order_ID
INNER JOIN Product p
ON oi.Product_ID = p.Product_ID
LEFT JOIN Payment pay
ON o.Order_ID = pay.Order_ID
ORDER BY o.Order_ID, p.Product_ID;
```

```
SQL> SELECT c.Customer_Name, o.Order_ID, p.Product_Name,
  2  oi.Quantity AS Qty,
  3  oi.Quantity * oi.Unit_Price AS Line_Total,
  4  pay.Payment_Method AS Pay_Method,
  5  pay.Payment_Status AS Pay_Status
  6  FROM Customer c
  7  INNER JOIN "Order" o
  8  ON c.Customer_ID = o.Customer_ID
  9  INNER JOIN Order_Item oi
 10  ON o.Order_ID = oi.Order_ID
 11  INNER JOIN Product p
 12  ON oi.Product_ID = p.Product_ID
 13  LEFT JOIN Payment pay
 14  ON o.Order_ID = pay.Order_ID
 15  ORDER BY o.Order_ID, p.Product_ID;

CUSTOMER_NAME ORDER_ID PRODUCT_NAME           QTY LINE_TOTAL PAY_METHOD  PAY_STATUS
------------- -------- ---------------------- --- ---------- ----------- ----------
Arun Kumar        5001 KALLAX Shelf Unit        1       4999 UPI         Paid
Arun Kumar        5001 LACK Coffee Table        2       4998 UPI         Paid
Arun Kumar        5002 FADO Table Lamp          2       2998 Credit Card Paid
Arun Kumar        5002 FEJKA Artificial Plant   3       2397 Credit Card Paid
Divya Lakshmi     5003 MALM Bed Frame           1      12999 Net Banking Paid
Karthik Raja      5004 MICKE Desk               1       5999 COD         Pending
Karthik Raja      5004 MULIG Clothes Rack       2       3998 COD         Pending
Meena Priya       5005 POANG Armchair           1       6999 UPI         Paid
Meena Priya       5006 KALLAX Shelf Unit        1       4999
Meena Priya       5006 APPLARO Outdoor Table    1       8999

10 rows selected.
```

8. JOIN COMPARISON

Join type	Rows returned	Treatment of unmatched rows	Example use in the IKEA database
INNER JOIN	Only rows with a match in both tables	Removed from the result	Order items with their product names
LEFT JOIN	All rows of the left table plus matches from the right	Left rows kept; right columns are NULL	All customers, including those with no orders

Join type	Rows returned	Treatment of unmatched rows	Example use in the IKEA database
RIGHT JOIN	All rows of the right table plus matches from the left	Right rows kept; left columns are NULL	All products, including those never ordered

9. CONCLUSION

This task demonstrated how joins retrieve connected information from the IKEA E-Commerce Database. INNER JOIN combined customers, orders, order items, products and payments, LEFT JOIN and RIGHT JOIN showed customers without orders, orders without payments and products never ordered, and aggregate queries produced order counts, product sales and customer purchase totals. These queries support order tracking, customer purchase history, payment checking and business reporting.